

Deep Learning and AI Summative

April 2026

1 Introduction

In this report, we discuss the approach to classification and generation of characters using neural networks. We perform classification of 10,000 images based on a training set of 500 labelled images. We then aim to generate 1000 new images similar to those within the dataset.

The data consists of 6 different categories of 28×28 pixel images, in which A and B are circle-based, C and D are line-based and E and F resemble a D shape. As a result, we expect that classification between these pairings (e.g. A vs E) will be relatively simple, but correctly classifying within pairings will be more challenging as, for example, E and F appear to be similar shapes differing only by whether they contain one or two dots inside. We observe that the training data set consists of 500 labelled images, with approximately 80 per class, which is a small dataset and so models may be at risk of overfitting. We note further that the training images all appear to be the same orientation, whereas the test images show rotated versions of the shapes, indicating that data augmentation may be useful to ensure models learn features rather than locations. While this observation technically means test data influenced model decisions, only visual characteristics were used and all data was unlabelled. We use categorical cross-entropy loss, as standard for multi-class classification. Pixel values were normalised by division by 255 as neural networks train more effectively with smaller numbers. The images are greyscale so use a single channel.



Figure 1: Example image from each category, where natural pairings A/B, C/D, E/F are visible.

2 Classification

2.1 Feed-Forward Neural Network (FFNN)

As a baseline, a feed-forward neural network was implemented. Although Cybenko's Universal Approximation Theorem guarantees that one hidden layer is theoretically sufficient, adding depth is more efficient for structured data such as images. A model with two hidden layers was implemented to allow feature combination while controlling model complexity, as the training data set is small and more complex models require more training samples by the Fundamental Theorem of Statistical Learning. Therefore, there is a trade-off between model capacity via a more complex model with more neurons, and overfitting. The validation set is made up of 20% of the data, with the remaining 80% used to train the model. A batch size of 32 is used throughout to balance computational speed with noise, since smaller batches update faster but introduce more noise.

We use ReLU activation functions in hidden layers ($\text{ReLU}(x) = \max(0, x)$), whose derivative is in $\{0, 1\}$. This mitigates the vanishing gradients problem exhibited by other activation functions such as the logistic function, where gradients shrink exponentially with depth. At the output layer, we use the softmax function which outputs probabilities that sum to 1. Under 0-1 loss, this enables us to set our estimate as the class with the highest probability. We compile with the 'RMSprop' optimiser, a variant of gradient descent which adapts the learning rate for each weight. Initial weights are set using Glorot initialisation within *keras*, which preserves the activation variance across layers, preventing exploding or vanishing gradients before training begins.

Grid search was used to tune the parameters number of neurons in each layer (32, 64, 128, 256) and dropout rate (0.1, 0.2, 0.3, 0.4), with the restriction that the number of neurons in layer 2 is less than that of layer 1. Low dropout rate was used to due to the small training set, to ensure that sufficient samples remained for training,

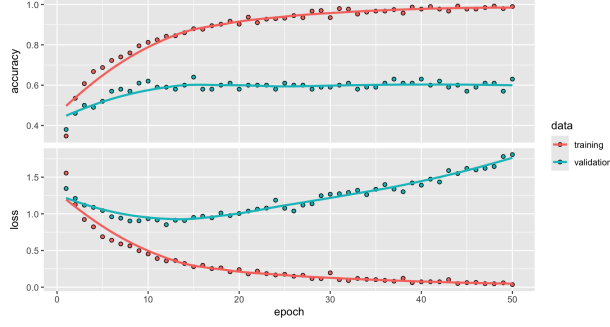
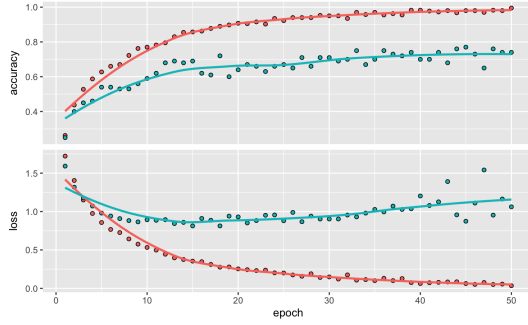
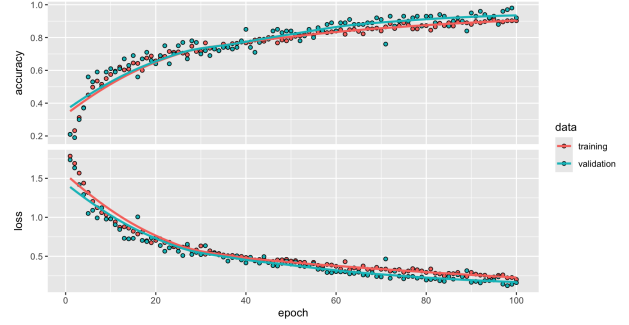


Figure 2: Training (red) and validation (blue) loss and accuracy for a feed-forward neural network with 128 neurons in the first layer, 64 in the second layer and a dropout rate of 0.1 in both layers.



(a) Training (red) and validation (blue) loss and accuracy for the CNN without augmentation.



(b) Training (red) and validation (blue) loss and accuracy for the CNN with augmentation.

Figure 3

while still preventing the network relying too heavily on any single neuron. In general larger networks with lower dropout performed best. The best accuracy was yielded by 128 neurons in the first layer, 64 neurons in the second layer and a dropout rate of 0.1, which had accuracy of 0.64, with the training curve shown in figure 2. Overfitting is demonstrated by the plateau of the validation accuracy, while the training accuracy continues to improve. We note that random classification would give an expected accuracy of 0.167, so this model is significantly better than random allocation.

2.2 Convolution Neural Network (CNN)

While the feedforward network performs reasonably well, it treats the 784 pixel values as independent inputs with no spatial relationship. This is inefficient as it ignores the structure of the data implied by spatial positioning, making the model fitting task more complicated than it needs to be. A convolutional neural network addresses this by considering positional information through learned kernels that slide across the image, detecting local spatial features such as edges and dots. This additionally benefits from weight sharing, where the same kernel weights are applied at every position. This means a convolutional layer requires far fewer parameters than a feed-forward network layer, which is advantageous given our small dataset and constraints on the model complexity as discussed above.

We use two 3×3 convolutions, with 2×2 maximum pooling at each step. In the first convolution we use 32 filters, and in the second we use 64. The number of filters increases at each step as pooling reduces the spatial dimensions, allowing the network to combine basic features from earlier layers into more complex ones. After flattening, we use a feed-forward neural network as above with 64 neurons to classify based on the extracted features, again using the softmax output function. This resulted in accuracy of 0.72, a significant improvement on the feed-forward neural network, as a result of exploiting the spatial relationship between pixels.

	A	B	C	D	E	F
A	15	3	0	0	0	0
B	0	14	0	0	0	0
C	0	0	12	3	0	0
D	0	0	6	11	0	0
E	0	0	0	0	13	11
F	0	0	0	0	2	10

(a)

	A	B	C	D	E	F
A	15	1	0	0	0	0
B	0	16	0	0	0	0
C	0	0	18	1	0	0
D	0	0	0	13	0	0
E	0	0	0	0	15	2
F	0	0	0	0	0	19

(b)

Table 1: Confusion matrix for CNN without augmentation (a) and with augmentation (b), where rows represent predicted class and columns represent true class.

True	Predicted	P(A)	P(B)	P(C)	P(D)	P(E)	P(F)
F	E	0	0	0	0	0.505	0.495
C	D	0	0	0.344	0.599	0.005	0.052
B	A	0.579	0.418	0.003	0	0	0
D	C	0	0.003	0.595	0.401	0	0

Table 2: Misclassified images from the validation set and their output probabilities from the augmented CNN model

2.3 CNN with Augmentation

In order to improve the model further, we add augmentation to the CNN, so that the model learns patterns rather than locations. As we have noted, the test images are rotated relative to the training images, so we add rotation by factor 0.05, translation both horizontally and vertically, and also horizontal and vertical flips. Since none of the classes are rotations, flips or translations of another class, all of these augmentations can be performed safely without introducing confusion. This resulted in a further improved model with accuracy of 0.96. Whereas we trained the other models for 50 epochs, we train the augmented CNN with 100 epochs, since augmentation slows learning as different image variations are used at each epoch.

We see from table 1 that the only mis-classifications made by the CNN models were between visually similar classes (A/B, C/D, E/F), with E and F the most confused in the non-augmentation case. This is improved by adding augmentation, though 4 images remain misclassified. Per-class accuracy improved across all classes with augmentation, with F improving most dramatically (48% to 91%). With augmentation, all classes achieved above 90% accuracy. The overall accuracy was 0.96, with the four mis-classified images being between visually similar pairs, where the certainty was relatively low in each case, as shown by table 2. The training curve is shown in figure 3b, noting that the training and validation curves follow each other much more closely than the FFNN and non-augmentation case, indicating a reduction in overfitting.

3 Generation

3.1 Variational Autoencoder (VAE)

To generate similar images, we first used a Variational Autoencoder (VAE). Combining the labelled and unlabelled images, we had a total of 10,500 training images and so we use a larger batch size of 128 to account for this. The encoder compresses the 784 pixels into a small latent space, and the decoder reconstructs images from points in that latent space. The encoder and decoder each use a single dense layer of 256 neurons. A standard autoencoder does not ensure that similar images map to nearby points in the latent space, whereas a VAE encodes images to a distribution rather than a single point. Constraining this distribution to be close to a standard normal through the use of the KL divergence penalty results in similar images in the original space being encoded to be close to one another in the latent space. The encoder outputs a mean and variance, from which a latent point is sampled using the reparameterisation trick and then passed to the decoder to reconstruct the image. VAE uses the *Adam* optimiser, which is adaptive and uses momentum to find global minima.

This was first implemented with 2 latent dimensions, producing images as shown in figure 4a. These images somewhat resemble the training images, however they are relatively blurry so that features aren't strongly distinguishable. This is a result of using a small number of latent dimensions, as 784 pixels must be constructed from only two latent dimensions which may not give sufficient information to capture features from all six categories. It

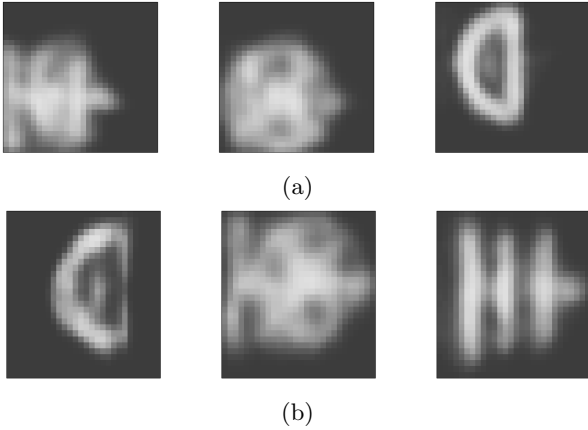


Figure 4: Example images generated by the 2D (a) and 10D (b) VAE model.

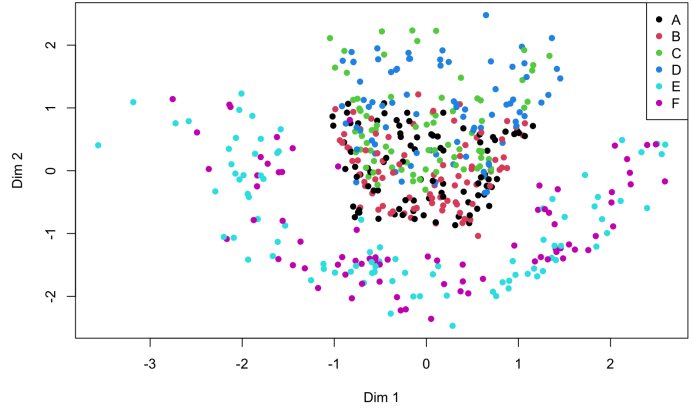


Figure 5: 2D latent space under VAE

captures general features, but loses finer details such as the presence of one or two dots which separates classes E and F. Moreover, in areas of uncertainty grey pixels are produced, adding to the blur effect, whereas the original images contain sharper boundaries. The plot of the latent space is shown in figure 5, where the pairs are grouped as expected, although there exists overlap between the A/B and C/D region, whereas the E/F region is more distinct. This indicates that the use of 2 dimensions is insufficient to capture the distinctions between classes.

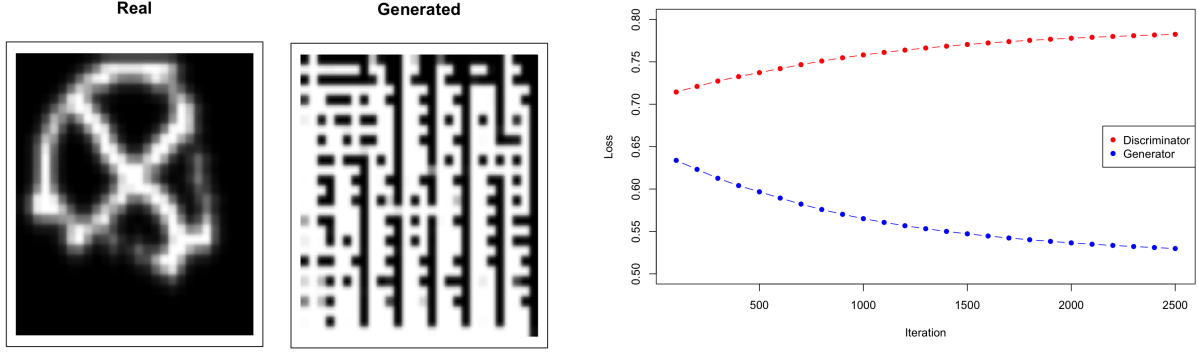
To quantitatively assess the quality of the generated images, we input them into the earlier CNN model, and assess the average model confidence in predicting a class. For images that do not distinctly resemble a single class, the model confidence will be low, since it will assign some probability across multiple classes. The images produced by the two dimensional latent space produced predictions only in classes B, D, E and F, such that A and C were not represented. It had a mean confidence of 0.77 across the data.

To improve this, we increase the dimension of the latent space to 10 to enable the model to represent more detailed features. The resulting images are shown in figure 4b. The mean minimum Euclidean distance between generated and real images was 968, with the closest case being 379. This confirms that the VAE produces novel images rather than memorising training data. Using the CNN model as above resulted in representation from all 6 classes, though A, C and E had significantly fewer assignments than B, D and F. Despite this, the confidence decreased to 0.74. This may be a result of high dimensionality, since sampling points in the 10-dimensional latent space which are too far from those represented by the training data produces uncertainty in the model. To mitigate this, we sample from random rows of the training data, and add on a small amount of noise so that the new points are close to the training points in the latent space. This results in an improved confidence of 0.80 and a more even spread of image assignment across classes. This had little impact on the average Euclidean distance (946.1), meaning that the images produced are distinct from the training data.

3.2 Generative Adversarial Network

An alternative approach for image generation is use of a Generative Adversarial Network (GAN), which consists of two neural networks: a generator, which aims to produce realistic images close to the training images from random noise, and a discriminator, which aims to distinguish between images produced by the generator and the original training images. Both use convolutions, and the weights are updated alternating between the generator and discriminator at each step. Whereas the VAE optimises a single loss function (a combination of the KL loss and the reconstruction loss), a GAN uses adversarial training, with two ‘competing’ losses. While the reconstruction loss can cause blurriness as grey pixels are output in regions of uncertainty, the GAN should produce sharper images.

After 2500 iterations, the images produced by the generator had grid-like artefacts as shown in figure 6a. Despite the visual dissimilarity, the discriminator loss was 0.77, while the generator loss was 0.54, which would indicate that the generator was outperforming the discriminator. The generator loss decreases more steeply than the discriminator loss increases (figure 6b), indicating that the generator learns faster than the discriminator. This is consistent with mode collapse, where the generator learns a repeated pattern that fools the discriminator, then with no incentive to learn the true image features. As a result, the VAE model was used as the outputs were visually more similar to the training images.



(a) Real training image (left) compared to the GAN generated image at iteration 2500 (right). (b) Loss of the discriminator (red) and generator (blue) sampled at each 100 iterations.

Figure 6

4 Other Models Considered

An alternative model that may have been effective on this data would be a diffusion model, which works by adding noise to a real image, then training a NN on removing this noise step by step. This may have produced less noisy images than the VAE due to the gradual noise reduction which introduces less uncertainty. The loss function is also simpler than adversarial loss in the GAN model, however it is difficult to implement and so was not attempted. A Boltzmann model, which learns a probability distribution over the input data, was also considered, however it is computationally expensive to implement. RNNs would not be applicable here as they work on sequential data such as text, and transfer learning would only be useful if we had a pre-trained data set on similar images, which is not the case for our random symbols.

5 Conclusion

For the classification task, the baseline feed-forward neural network produced an accuracy of 0.64, significantly better than random allocation. This was improved upon by exploiting spatial dependence with a CNN which had accuracy of 0.72. Adding augmentation improved this further to 0.96, with misclassified images having significant uncertainty as shown in table 2. In general, all classification methods successfully classified the between-pair classes, distinguishing between the circle, D-shape and line-based images, however the more complex CNN with augmentation was required to identify the finer details of the within-pair distinction.

For the image generation the VAE model with 2D latent space produced images visually similar to the training set, although they contained extra noise. The plot of the latent space highlighted mixing between classes and so a 10D model was used. When combined with samples from near training data, this offered improved image definition by considering more detailed features, and it was confirmed that the images were not copies of the original training set by comparing Euclidean distance. Though the GAN model would be applicable to this problem in theory, in practice its results were poor due to the issue of mode collapse.

A limitation here is the small data set, and more training data would likely lead to better results in both classification and generation. Further parameter tuning could be explored, for example the tuning of standard deviation in the normal sampling under the VAE model. All results presented are subject to stochastic variation of runs.